

PRACTICA 2- DIVIDE Y VENCERÁS

Subgrupo 2 del C2:

Hugo Aguiar García 20%

Jose Daniel Galán Gómez 20%

Alfonso Maldonado González 20%

.Adrián Rosado Rueda 20%

Felipe Ocaña Valenzuela 20%

ÍNDICE

PROBLEMA 1.1: AUDITORÍA FINANCIERA.....	2
Algoritmo Obvio $O(n^2)$.....	2
Eficiencia Teórica:.....	2
Explicación del Algoritmo:.....	2
Gráfica de Eficiencia Híbrida:.....	2
Primer Algoritmo Divide y Vencerás $O(n \log n)$.....	3
Eficiencia Teórica:.....	3
Explicación del Algoritmo:.....	3
Gráfica de Eficiencia Híbrida:.....	3
Segundo Algoritmo Divide y Vencerás $O(n \log n)$.....	4
Eficiencia Teórica:.....	4
Explicación del Algoritmo:.....	4
Gráfica de Eficiencia Híbrida:.....	4
Análisis de los 3 Algoritmos.....	5
Gráfica Comparativa de los 3 Algoritmos:.....	5
Análisis de la Gráfica:.....	5
Gráfica Comparativa de los 2 Algoritmos basados en Divide y Vencerás:.....	6
Análisis de la Gráfica:.....	6
PROBLEMA 2: MERGESORT Y QUICKSORT.....	7
Explicación algoritmos Mergesort:.....	7
Explicación algoritmos Quicksort:.....	7
Diferentes tipos de datos en quicksort:.....	8
Diferentes tipos de datos en mergesort:.....	9
Comparativa de los algoritmos con datos enteros:.....	10
Comparativa de ambos algoritmos con strings pequeños:.....	11
Comparativa de ambos algoritmos con strings grandes:.....	12
Estudio de los distintos umbrales para Mergesort con strings:.....	13
Estudio de los distintos umbrales para Quicksort con strings:.....	15
Estudio de los distintos umbrales para Mergesort con ints:.....	17
Estudio de los distintos umbrales para Quicksort con ints:.....	19

PROBLEMA 1.1: AUDITORÍA FINANCIERA

Algoritmo Obvio $O(n^2)$

Eficiencia Teórica:

Este algoritmo tiene una eficiencia teórica de $O(n^2)$ puesto que consta en el peor de los casos de dos bucles **for** anidados, donde el primero recorre el vector entero de **inicio** a **fin**, y, para cada elemento **i**, el segundo recorre el resto del vector desde **j = i + 1**. Por tanto el número de comparaciones crece proporcionalmente a n^2 .

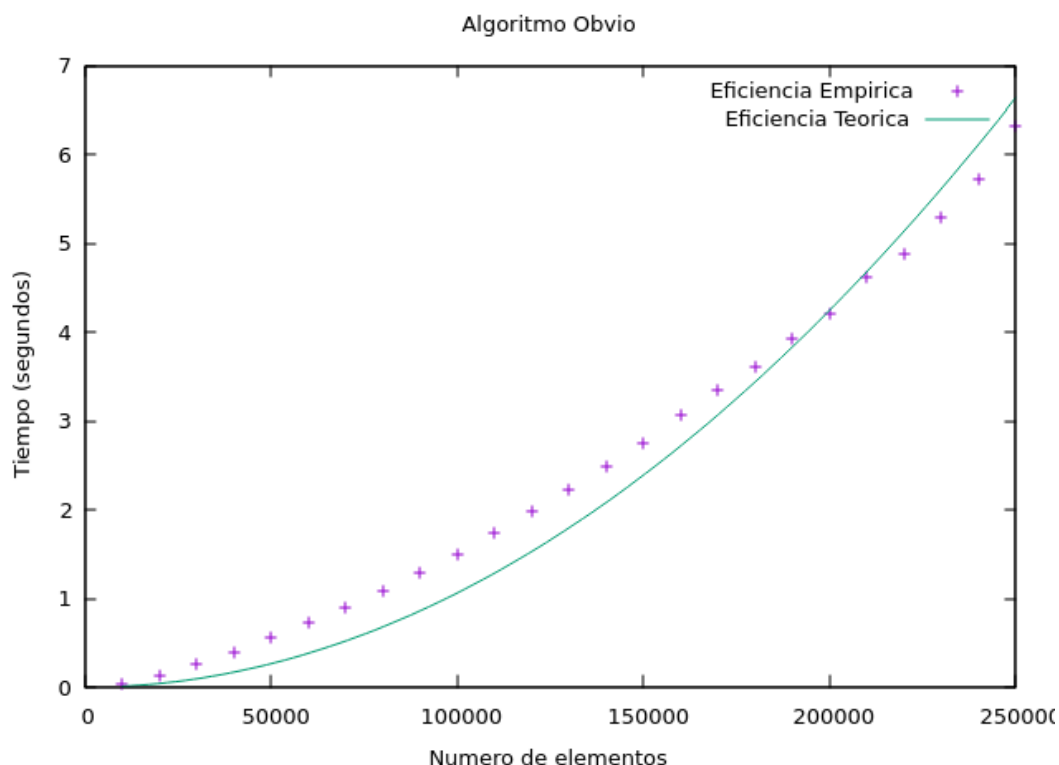
Explicación del Algoritmo:

En primer lugar, se comprueba si el vector **a** tiene al menos dos elementos. Si **a.size() < 2**, se devuelve directamente el vector **resultado** vacío, ya que no puede existir ninguna pareja.

Si el vector tiene suficientes elementos, se fijan las variables **inicio = 0** y **fin = a.size() - 1**. A continuación, se recorre el vector con un primer bucle usando la variable **i**. Para cada transacción **a[i]**, se calcula su **complemento**, es decir, el valor que haría falta para que la suma de ambas transacciones fuese igual al objetivo **X**.

Después, un segundo bucle recorre el vector desde la posición siguiente, **j = i + 1**, hasta **fin**, buscando ese **complemento**. De esta forma se evita comparar una transacción consigo misma y también repetir parejas ya estudiadas. Si **a[j]** coincide con el **complemento**, se crea una nueva estructura **par**, se guardan en ella los valores encontrados y se añade al vector **resultado**. Tras eso, se continúa con la siguiente posición del primer bucle.

Gráfica de Eficiencia Híbrida:



Primer Algoritmo Divide y Vencerás $O(n \log n)$

Eficiencia Teórica:

Para esta implementación hemos implementado el famoso código de ordenación **Quicksort** (de media su eficiencia teórica es de $O(n \log n)$ y en su peor caso $O(n^2)$) junto con un algoritmo de búsqueda binaria recursiva ($O(\log n)$) lo que nos da un total de un algoritmo de eficiencia teórica de ($O(n \log n)$) salvo en el peor caso que será $O(n^2)$.

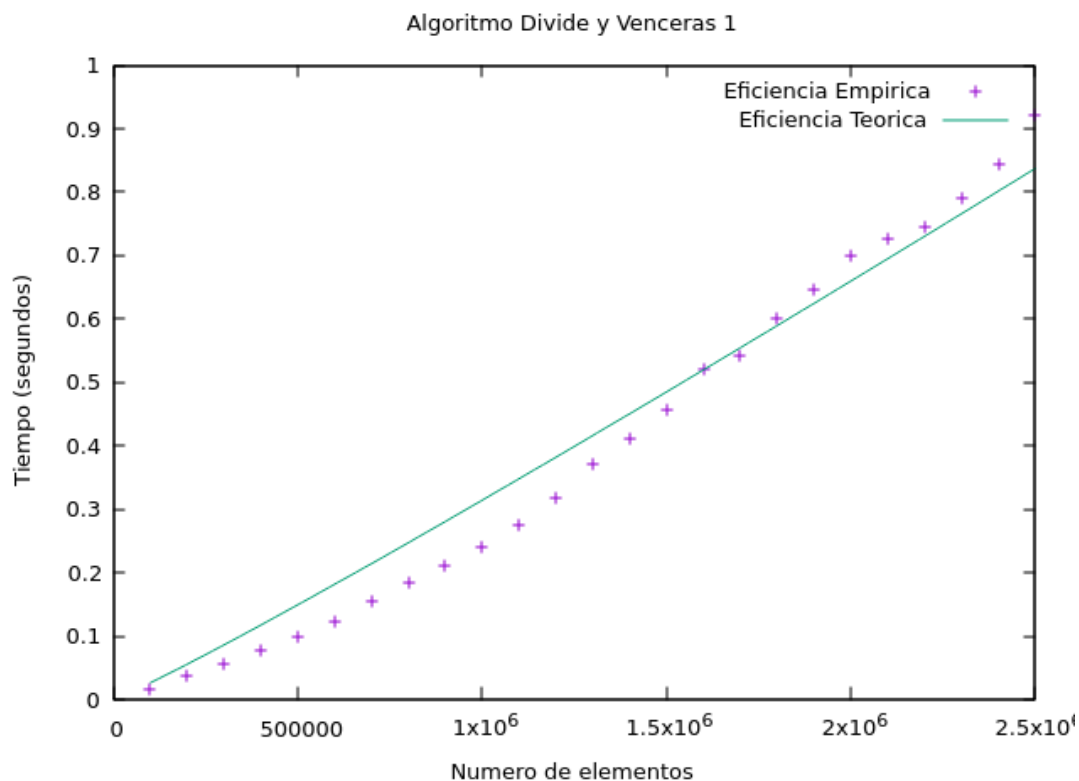
Explicación del Algoritmo:

Al igual que en el algoritmo anterior, primero se comprueba si el vector **a** tiene al menos dos elementos. Si no los tiene, se devuelve **resultado** vacío. En caso contrario, se fijan **inicio** y **fin** y se llama a **quicksort(a, inicio, fin)** para ordenar el vector.

La ordenación se realiza mediante la función **particion**, que toma como pivote **a[inicio]** y usa los índices **izq** y **der** para recolocar los elementos menores a un lado y los mayores al otro. Cuando el pivote queda en su posición correcta, el proceso se repite recursivamente sobre las dos partes generadas.

Una vez ordenado el vector, se recorre con un bucle usando **i**. Para cada elemento **a[i]**, se calcula su **complemento** y se busca en la parte derecha del vector mediante **busquedaBin(a, complemento, i + 1, fin)**. Si la función devuelve una posición distinta de **-1**, se guarda la pareja en **resultado**.

Gráfica de Eficiencia Híbrida:



Segundo Algoritmo Divide y Vencerás $O(n \log n)$

Eficiencia Teórica:

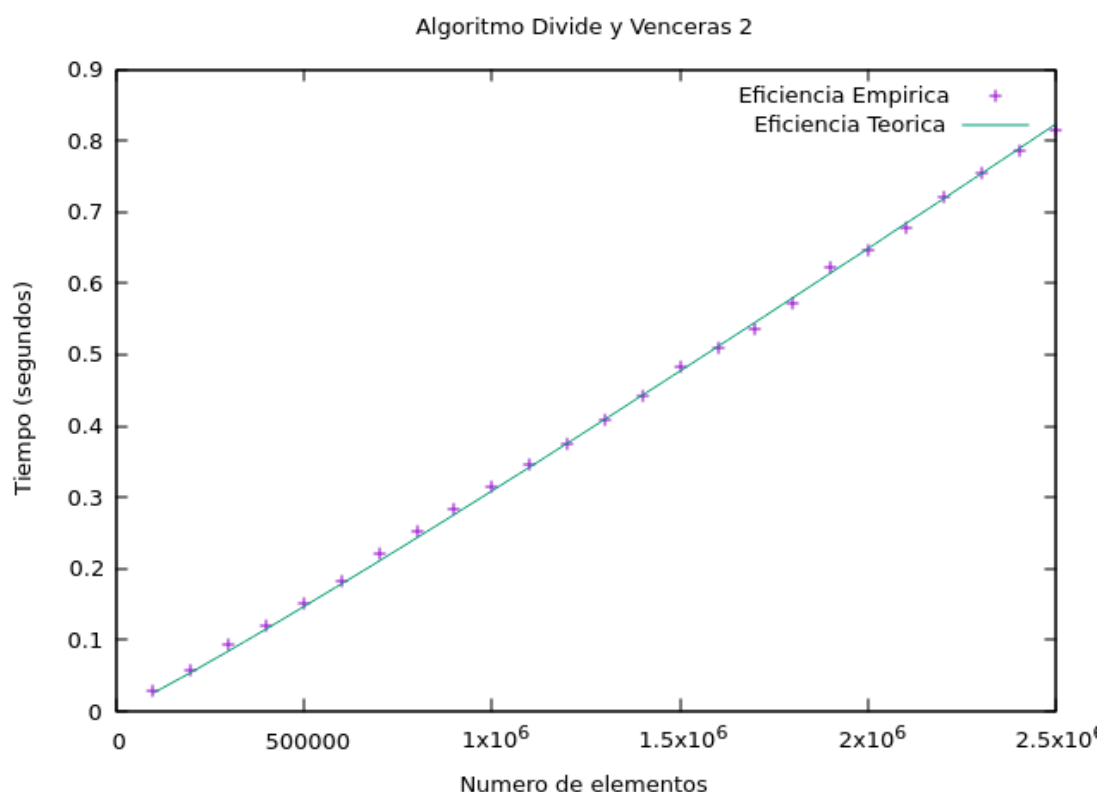
Para esta implementación hemos utilizado el algoritmo de ordenación **Mergesort** (de eficiencia teórica $O(n \log n)$ en todos los casos), junto con la técnica de **dos punteros** para buscar las parejas que suman el valor objetivo **X** en tiempo $O(n)$. Por tanto, la eficiencia teórica total del algoritmo es $O(n \log n)$, ya que el coste dominante es el de la ordenación.

Explicación del Algoritmo:

Al igual que en el caso anterior, comenzamos comprobando que el vector **a** tiene al menos dos elementos. Si es así, lo primero que hacemos es ordenarlo con **Mergesort**, llamando a **mergesort(a, 0, a.size() - 1)**. La función **mergesort** divide el vector en dos partes usando **medio**, ordena recursivamente cada mitad y después las une mediante **merge**, apoyándose en los vectores auxiliares **izq** y **der**.

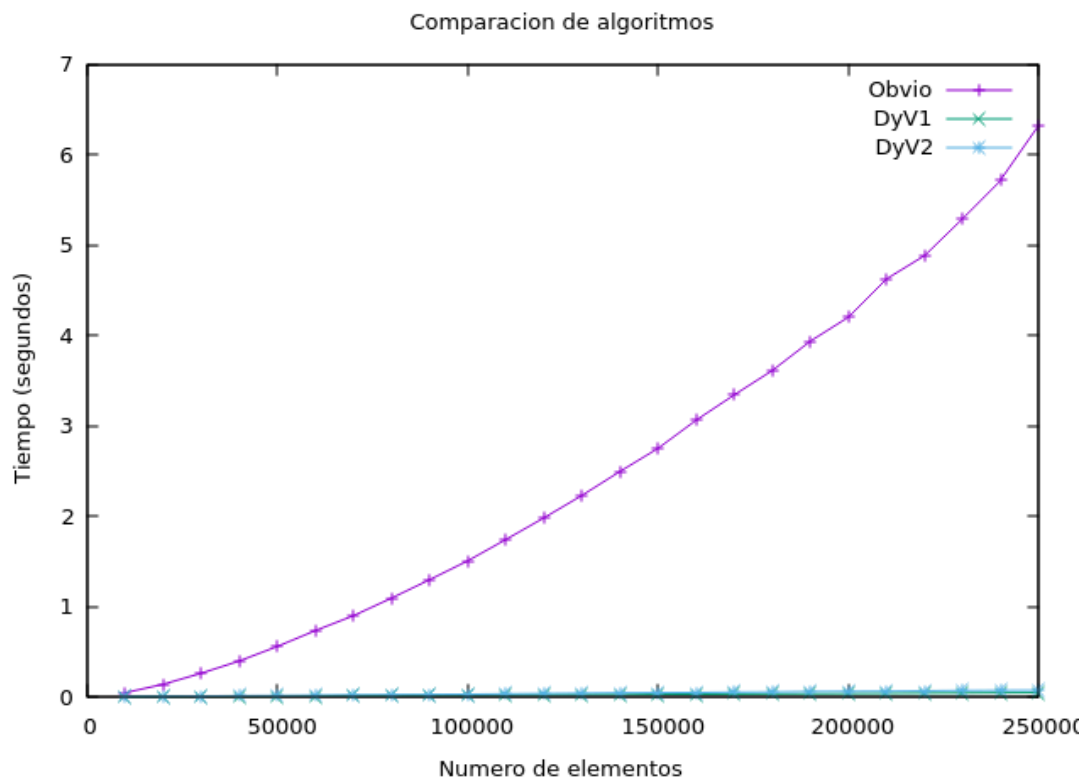
Una vez ordenado el vector, utilizamos la técnica de los **dos punteros**: colocamos uno **i** al inicio del vector y otro, el puntero **j** al final. Mientras **i < j**, se calcula la **suma = a[i] + a[j]**. Si la suma coincide con el objetivo **X**, se guarda la pareja y ambos punteros avanzan hacia el centro. En caso de que la suma sea menor, el **puntero izquierdo** se mueve hacia la derecha para aumentar la suma (se incrementa **i**), y en caso contrario, es decir, que la suma sea mayor, se mueve el **puntero derecho** hacia la izquierda para disminuirla (se decrementa **j**). Así se van encontrando las parejas de forma eficiente sin necesidad de buscar elemento por elemento.

Gráfica de Eficiencia Híbrida:



Análisis de los 3 Algoritmos

Gráfica Comparativa de los 3 Algoritmos:



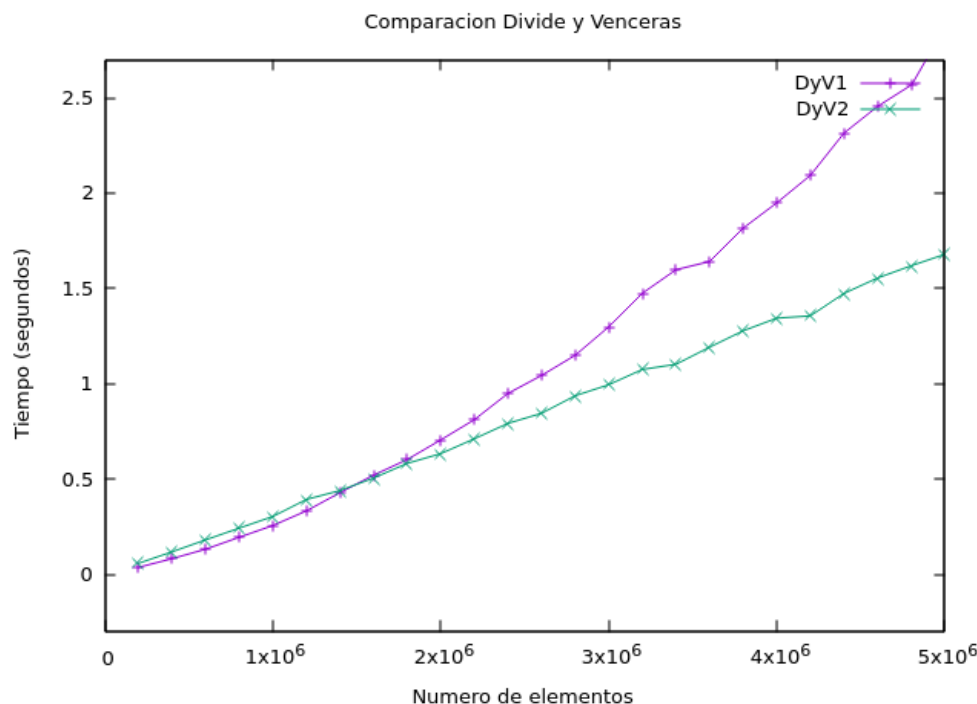
Análisis de la Gráfica:

Podemos observar cómo efectivamente, ambos algoritmos basados en la técnica de **Divide y Vencerás** son mucho más eficientes que el **Algoritmo Obvio**. Aunque al inicio las diferencias entre los tres algoritmos son mínimas, siendo el tiempo que tardan con **10.000 elementos** prácticamente el mismo tiempo (milésimas de segundo).

Conforme el **número de elementos** que hay en el vector aumenta progresivamente, vemos que la gráfica del **Algoritmo Obvio** crece de forma cuadrática, alejándose cada vez más de los tiempos de los otros algoritmos, tardando con 250.000 elementos este algoritmo de eficiencia teórica $O(n^2)$ poco más de 6 segundos, mientras que los dos algoritmos basados en **Divide y Vencerás** con una eficiencia teórica de $O(n \log n)$, no llegan a tardar siquiera a la cuarta parte de un segundo.

Pese a que se ve con claridad la diferencia abismal de tiempos entre el **Algoritmo Obvio** y los basados en **Divide y Vencerás**. No se aprecia con claridad el funcionamiento de los algoritmos de eficiencia $O(n \log n)$, así que vamos a hacer una gráfica con estos dos algoritmos y analizar sus comportamientos conforme el número de elementos aumenta.

Gráfica Comparativa de los 2 Algoritmos basados en Divide y Vencerás:



Análisis de la Gráfica:

En este caso, la gráfica aumenta en **200.000 elementos** cada vez, llegando a **5.000.000 elementos**. Podemos observar que efectivamente ambos algoritmos son muy similares en cuanto a rendimiento, y que ni siquiera llegan a tardar un segundo en la búsqueda de 2 elementos que sumados obtengan **X** con 20 veces más elementos que el algoritmo cuadrático.

Sin embargo, pese a ser algoritmos con la misma **eficiencia teórica**, los tiempos de ejecución en cada caso son ligeramente diferentes. Vemos que cuanto menor es el número de elementos, el algoritmo basado en **Quicksort** funciona mejor que el algoritmo basado en **Mergesort**, pero a partir de que el vector contenga 1.5 millones de elementos, el segundo algoritmo es más eficiente que el primero.

Esto se debe, en primer lugar, a que **Mergesort** mantiene siempre un comportamiento regular de orden **$O(n \log n)$** , mientras que **Quicksort** depende de cómo se realice la partición del vector. En esta implementación, el pivote se escoge como el primer elemento, **a[inicio]**, lo que puede generar particiones poco equilibradas y empeorar el rendimiento, llegando incluso al peor caso **$O(n^2)$** .

Y no solo eso, sino que en el primer algoritmo, tras ordenar con **Quicksort**, se realiza una **búsqueda binaria**. En cambio, en el segundo algoritmo, después de ordenar con **Mergesort**, la búsqueda de parejas se hace mediante la técnica de **dos punteros**, que recorre el vector de forma lineal y con menor sobrecarga.

PROBLEMA 2: MERGESORT Y QUICKSORT

Explicación algoritmos Mergesort:

Mergesort es un algoritmo de división estática que basa su eficiencia en la fase de combinación.

Divide el vector en dos mitades exactas de forma recursiva hasta llegar a subvectores de tamaño 1 (o menores al umbral establecido). Una vez divididos, utiliza un proceso de mezcla (merge) que recorre los elementos comparándolos para reconstruir el vector de forma ordenada.

Su complejidad temporal es siempre $O(n \log n)$ en todos los casos (mejor, peor y promedio), ya que la división siempre es equilibrada.

Explicación algoritmos Quicksort:

A diferencia de **Mergesort**, **Quicksort** es un algoritmo de división dinámica, donde el peso del trabajo recae en la fase de partición.

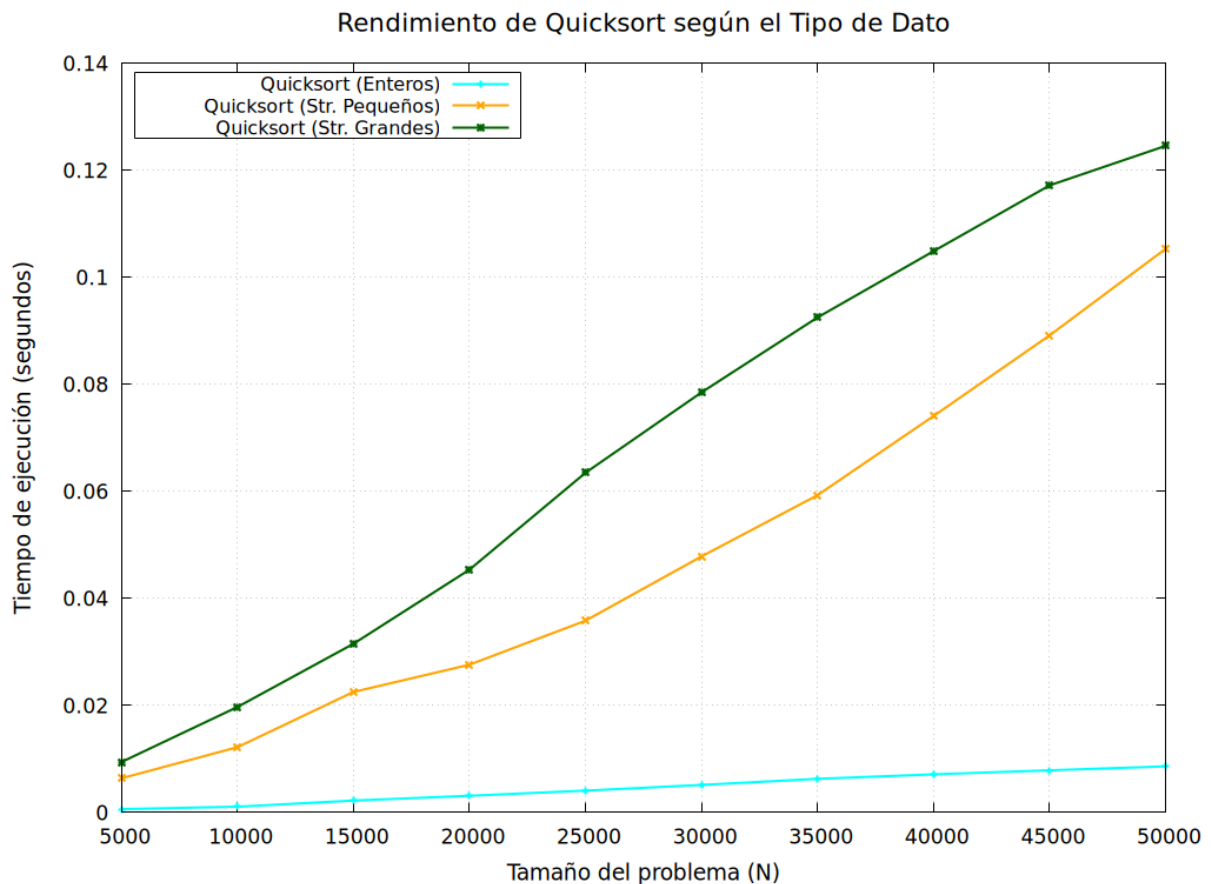
Selecciona un elemento llamado pivote y reorganiza el vector de modo que los elementos menores al pivote queden a su izquierda y los mayores a su derecha. Luego, aplica el mismo proceso recursivamente a las dos particiones generadas.

Su complejidad promedio es $O(n \log n)$, aunque en el peor caso (pivote muy mal elegido en vectores ya ordenados o casi ordenados) puede degradarse a $O(n^2)$.

Para mejorar el rendimiento de ambos algoritmos, se ha implementado una técnica de optimización que consiste en detener la recursión cuando el tamaño del subproblema es pequeño.

Para vectores de tamaño reducido ($n < U$), el algoritmo de ordenación por inserción suele ser más rápido que la sobrecarga que genera la recursión y las particiones de los algoritmos avanzados.

Diferentes tipos de datos en quicksort:

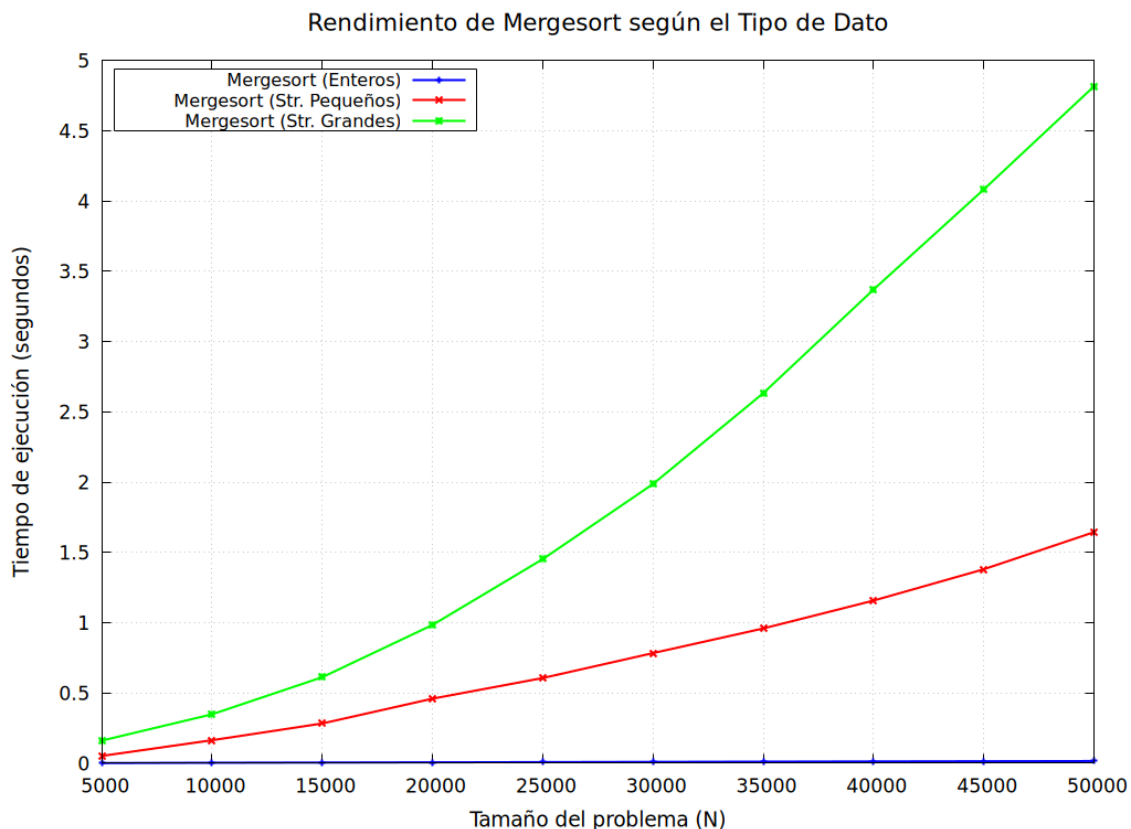


Esta gráfica muestra el tiempo de ejecución del algoritmo Quicksort aislando la variable del tipo de dato, comparando específicamente enteros, strings pequeños y strings grandes con un prefijo común. Se observa que la curva inferior, correspondiente a la ordenación de números **enteros**, se mantiene prácticamente **plana** y por debajo de los 0,01 segundos para un tamaño de hasta N=50.000. En contraste, las dos curvas superiores revelan un **incremento drástico** en el tiempo de ejecución al emplear **strings**.

Desde un punto de vista teórico, aunque el algoritmo mantiene una complejidad de $O(N \log N)$ en su caso promedio, el coste computacional de cada comparación individual varía significativamente según la naturaleza de los datos procesados. Mientras que la comparación de enteros es una operación atómica a nivel de hardware que requiere un único ciclo de reloj, la comparación de objetos string obliga al operador a acceder a la memoria dinámica e iterar sobre los caracteres.

En conclusión, estos resultados demuestran que el tipo de dato es un factor determinante que multiplica el coste base del algoritmo, aunque **Quicksort** mantiene su robustez general gracias a su capacidad de realizar la ordenación

Diferentes tipos de datos en mergesort:



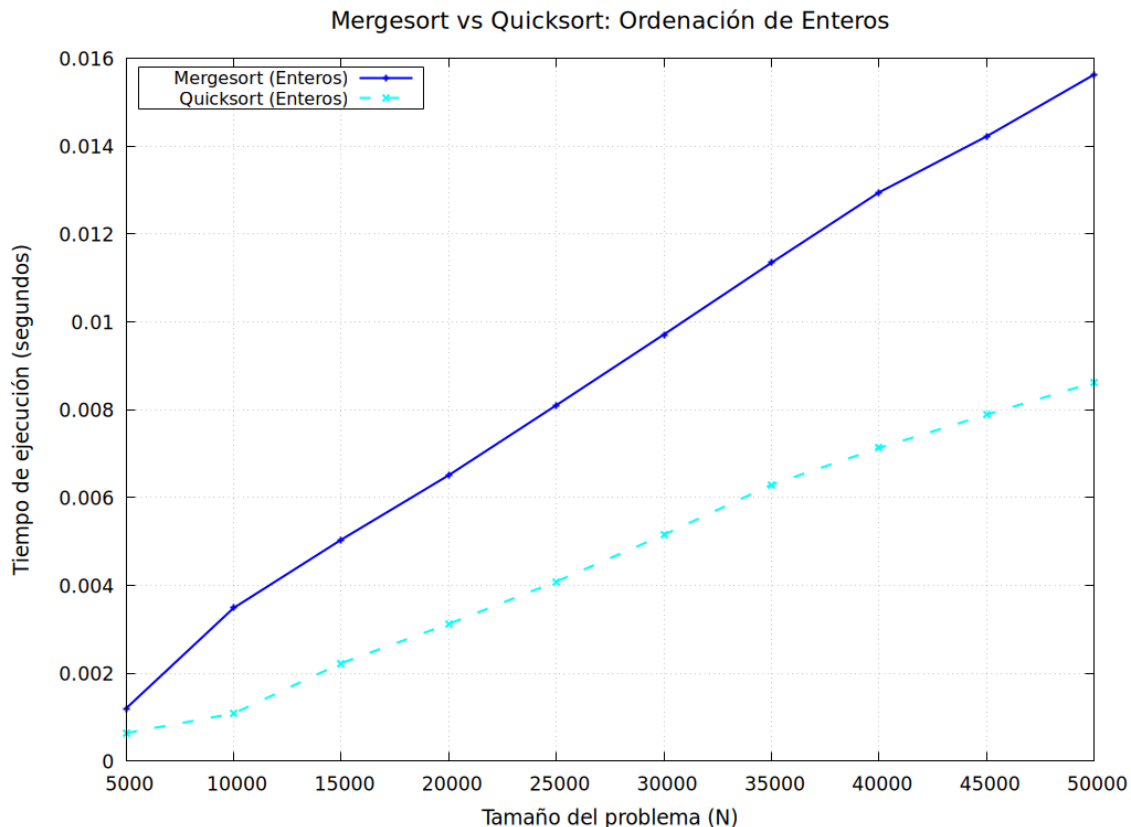
Esta gráfica presenta el rendimiento del algoritmo **Mergesort** bajo las mismas tres condiciones de datos, evidenciando una sensibilidad mucho más acentuada al tipo de información procesada que en el caso anterior. Mientras que la ordenación de enteros se mantiene de forma constante cerca del eje de abscisas debido a su bajo coste operativo, los tiempos para los tipos string se disparan de manera notable. En particular, la curva de strings grandes con prefijo común (verde brillante) muestra un crecimiento exponencialmente más agresivo, alcanzando casi los 5 segundos para un tamaño de $N=50.000$, lo que supone una diferencia de magnitud considerable frente a los strings pequeños (rojo), que se sitúan en torno a los 1,6 segundos.

La razón técnica de este comportamiento reside en la propia naturaleza del algoritmo de mezcla. A diferencia de **Quicksort**, **Mergesort** no es un algoritmo *in-situ*; requiere la creación constante de vectores auxiliares y la copia de elementos durante la fase de fusión.

Este coste de gestión de memoria, sumado a la lentitud de las comparaciones carácter a carácter en cadenas con prefijos idénticos, explica por qué **Mergesort** presenta tiempos tan elevados en esta gráfica. El impacto del tipo de dato aquí no solo afecta a la comparación lógica, sino también a la eficiencia del movimiento de datos en la memoria RAM.

En resumen, **Mergesort** demuestra una penalización mucho mayor al escalar hacia tipos de datos complejos y de gran tamaño, debido a la sobrecarga que supone su necesidad de espacio auxiliar y la duplicación de objetos pesados durante el proceso de ordenación.

Comparativa de los algoritmos con datos enteros:

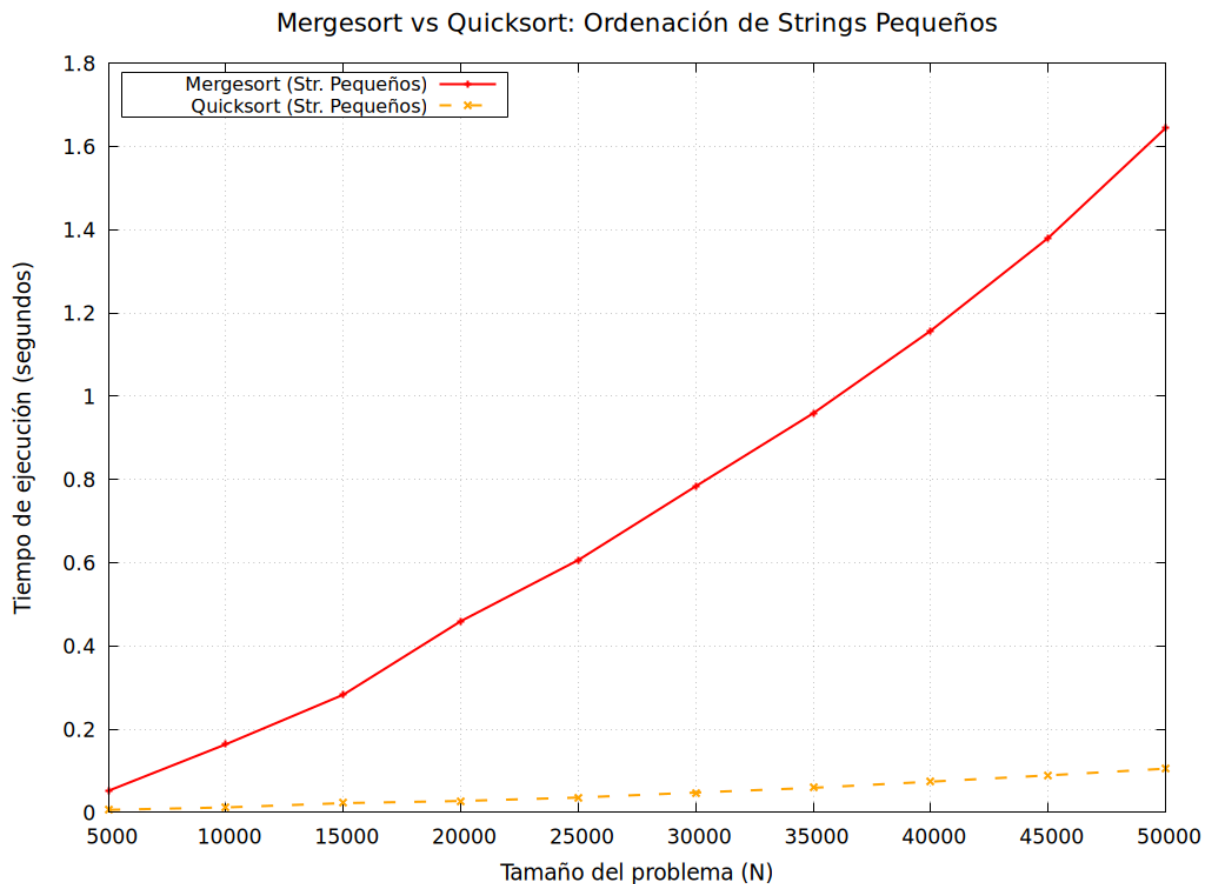


Esta gráfica muestra la comparativa directa entre **Mergesort** y **Quicksort** utilizando exclusivamente números enteros como tipo de dato. Al observar las curvas, se aprecia que ambos algoritmos presentan un crecimiento muy eficiente, manteniéndose en tiempos extremadamente bajos que no superan los 0,016 segundos para el tamaño máximo de $N=50.000$. Sin embargo, **Quicksort** (línea punteada cian) logra una ventaja competitiva constante sobre Mergesort (línea continua azul), situándose prácticamente a la mitad del tiempo de ejecución en casi todos los puntos de la muestra.

Aunque ambos algoritmos comparten una complejidad teórica de orden $O(n \log n)$ para este caso, la sobrecarga que supone para el sistema operativo gestionar la memoria dinámica en **Mergesort** marca la diferencia visible en la gráfica. Al trabajar con enteros, que son datos ligeros, esta penalización por la gestión de memoria se convierte en el factor principal que hace que **Mergesort** sea sistemáticamente más lento que **Quicksort** en esta comparativa específica.

En conclusión, para tipos de datos primitivos donde la comparación es inmediata, la eficiencia en el manejo de la memoria y la localidad de datos de **Quicksort** le otorgan una superioridad empírica clara.

Comparativa de ambos algoritmos con strings pequeños:

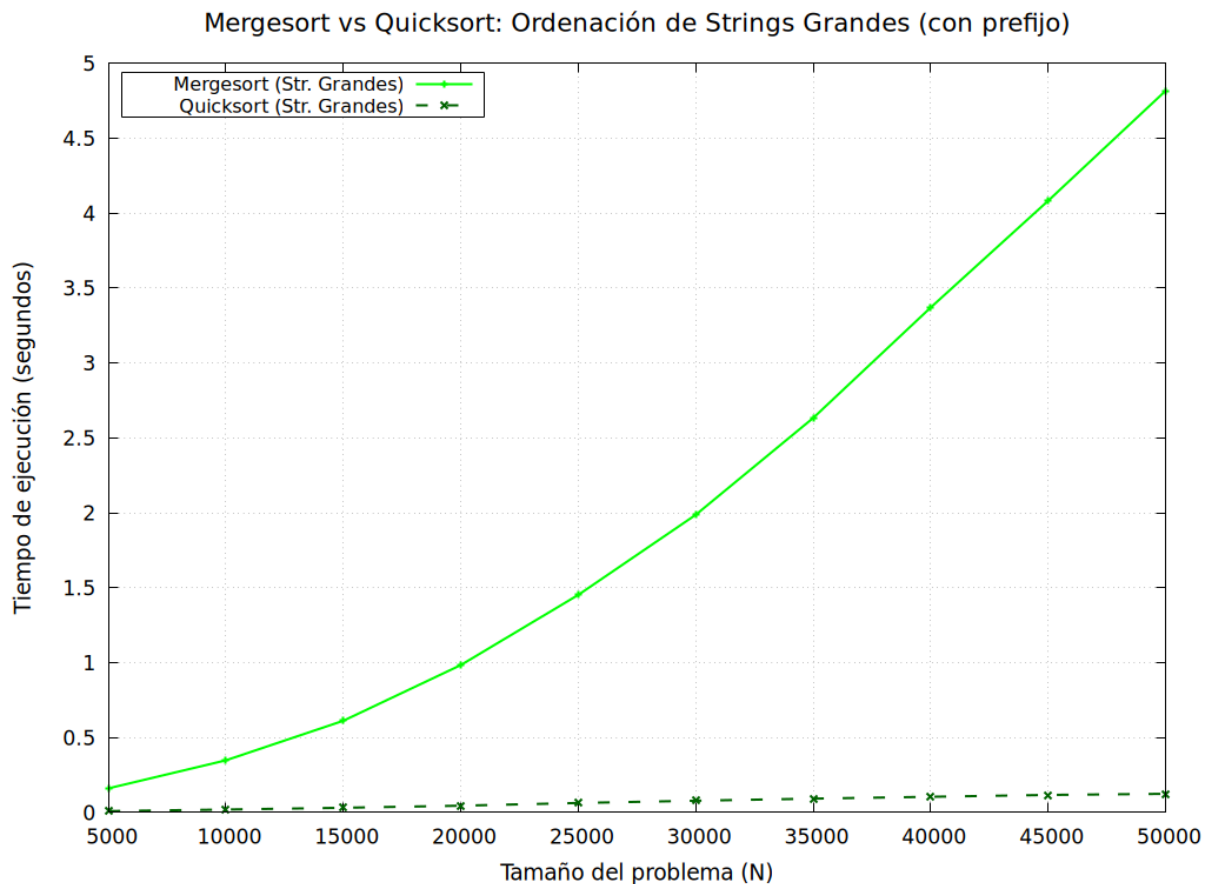


Esta gráfica compara el rendimiento de **Mergesort** y **Quicksort** al ordenar vectores de strings de tamaños pequeños y variables (longitudes entre 5 y 40 caracteres con prefijos cortos). Se observa una divergencia masiva entre ambos algoritmos: mientras que Quicksort (línea naranja discontinua) mantiene un crecimiento muy controlado, situándose apenas por encima de los 0,1 segundos para $N=50.000$, Mergesort (línea roja continua) muestra una pendiente mucho más pronunciada, escalando hasta los 1,64 segundos para el mismo volumen de datos.

La causa técnica de esta disparidad radica en la gestión de memoria y el coste de manipulación de objetos. Al ordenar strings, Mergesort debe realizar copias completas de estos objetos en vectores auxiliares durante cada fase de fusión. Mover un string implica no sólo copiar el puntero, sino gestionar la memoria dinámica asociada a cada cadena, lo que genera una sobrecarga enorme comparada con el intercambio de posiciones que realiza Quicksort.

En conclusión, para vectores de objetos de tamaño variable como strings, la eficiencia de Quicksort al minimizar los movimientos de datos y evitar el uso de memoria auxiliar le otorga una ventaja competitiva decisiva frente a la estructura más rígida y demandante de Mergesort.

Comparativa de ambos algoritmos con strings grandes:

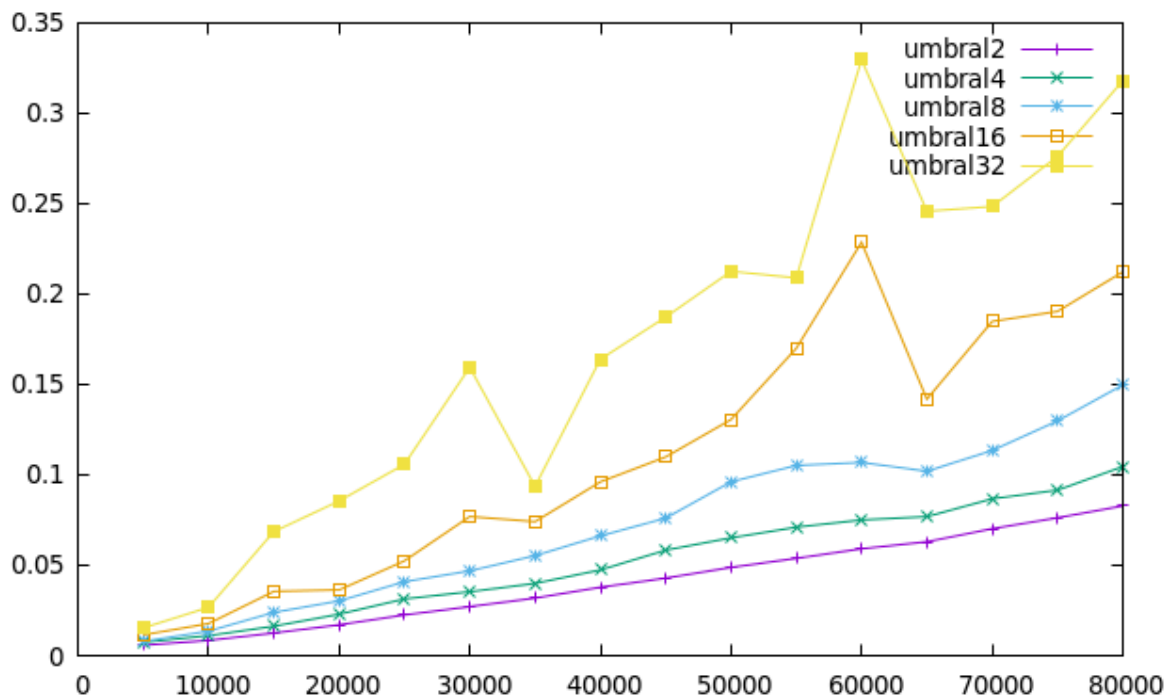


Esta última gráfica representa el escenario más crítico del estudio empírico: la ordenación de strings de gran tamaño (hasta 400 caracteres) con prefijos comunes proporcionalmente grandes. En este caso, la brecha de rendimiento entre los dos algoritmos alcanza su punto máximo, con Mergesort (línea verde continua) disparándose hasta casi los 5 segundos, mientras que Quicksort (línea verde punteada) se mantiene extraordinariamente eficiente por debajo de los 0,13 segundos para un tamaño de $N=50.000$. Esta diferencia visual tan extrema subraya cómo el diseño de cada algoritmo responde ante datos que combinan comparaciones costosas con estructuras de datos pesadas.

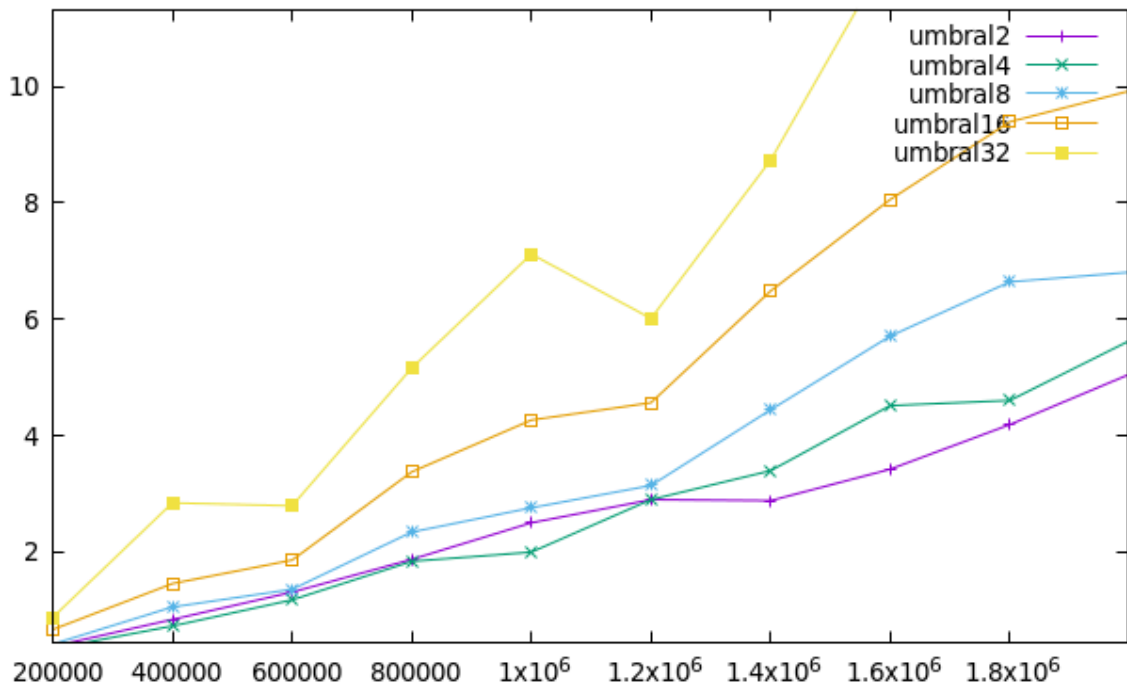
El factor determinante en esta gráfica es la combinación del alto coste de comparación y el alto coste de movimiento de datos. Al existir prefijos comunes muy largos, el operador de comparación debe recorrer cientos de caracteres idénticos antes de encontrar una diferencia, lo que eleva el tiempo de CPU en cada paso del algoritmo. Mergesort se ve doblemente penalizado porque, además de realizar estas comparaciones lentas, debe copiar repetidamente estas cadenas largas en vectores auxiliares durante la fase de mezcla, lo que genera una enorme sobrecarga en la gestión de memoria dinámica y el ancho de banda de la RAM.

En conclusión, esta comparativa final permite determinar que para tipos de datos complejos y de gran tamaño, la eficiencia estructural de Quicksort lo convierte en la opción más robusta, mientras que el modelo de "mezcla con memoria auxiliar" de Mergesort escala de forma muy ineficiente ante la combinación de datos pesados y comparaciones lentas.

Estudio de los distintos umbrales para Mergesort con strings:

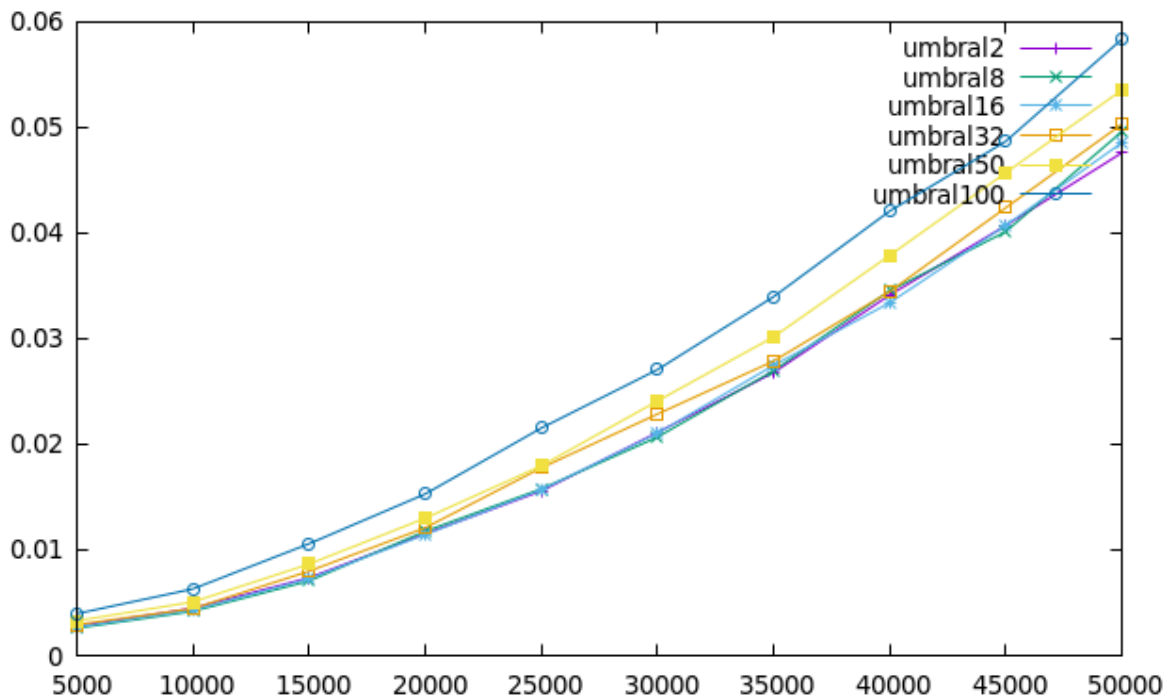


Esta gráfica muestra el comportamiento del algoritmo Mergesort al aplicar distintos umbrales (2, 4, 8, 16 y 32) en problemas de tamaño reducido, utilizando strings pequeños. Se observa que las diferencias entre los distintos umbrales no son excesivamente pronunciadas en los tamaños más pequeños, ya que el coste total del algoritmo sigue siendo bajo en todos los casos. Sin embargo, incluso en este rango, se aprecia que los umbrales más bajos (2 y 4) tienden a ofrecer mejores tiempos de ejecución. Esto se debe a que el algoritmo de inserción, aunque eficiente para tamaños muy pequeños, empieza a introducir cierto coste adicional cuando se aplica a subproblemas ligeramente mayores. Por otro lado, los umbrales más altos, especialmente 16 y 32, muestran un crecimiento más irregular y en general superior al resto. Esto indica que, incluso en problemas pequeños, aplicar inserción sobre subproblemas relativamente grandes no resulta óptimo, debido a su complejidad cuadrática.

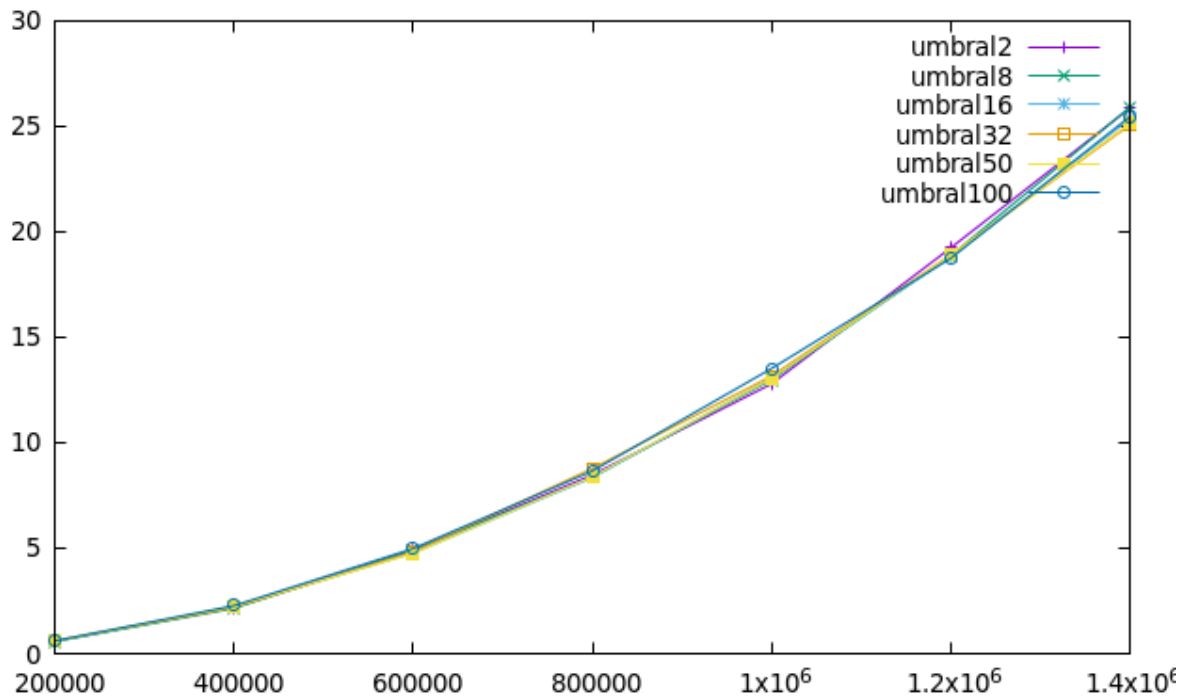


A medida que aumenta el tamaño del problema, las diferencias entre los distintos umbrales se vuelven mucho más evidentes. Los umbrales pequeños, como 2 y 4, presentan el crecimiento más controlado, manteniendo tiempos de ejecución significativamente inferiores al resto. Esto confirma que es preferible limitar el uso del algoritmo de inserción a subproblemas muy reducidos, dejando que Mergesort gestione la mayor parte del trabajo mediante su estrategia de divide y vencerás.

Estudio de los distintos umbrales para Quicksort con strings:



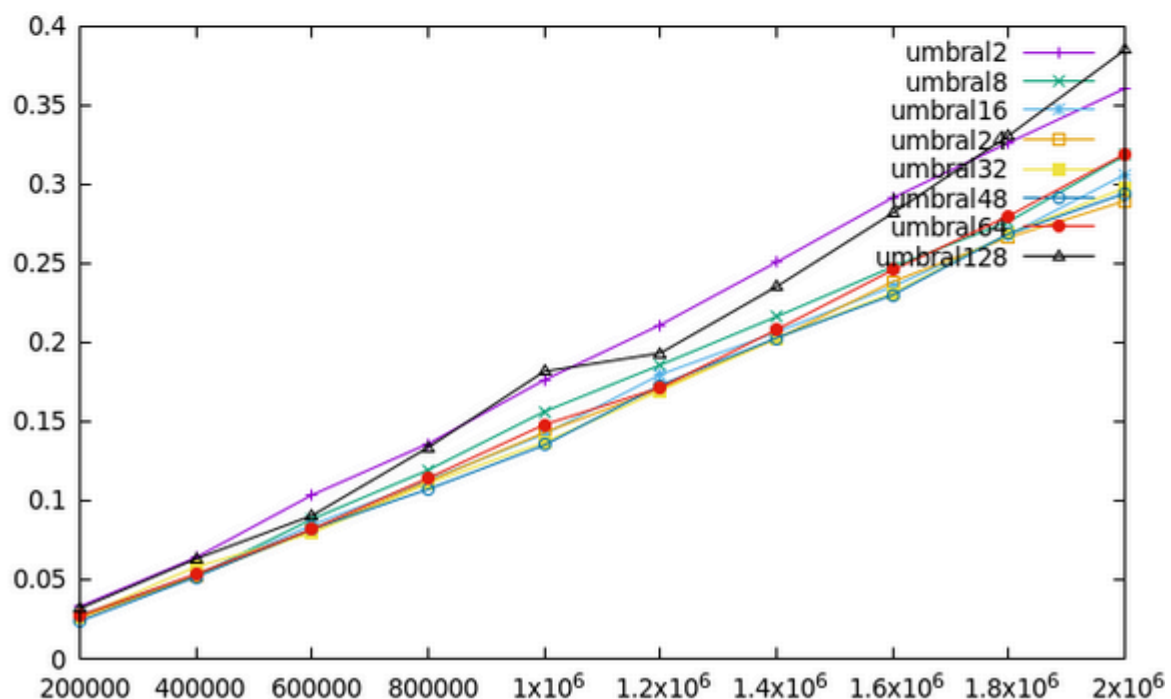
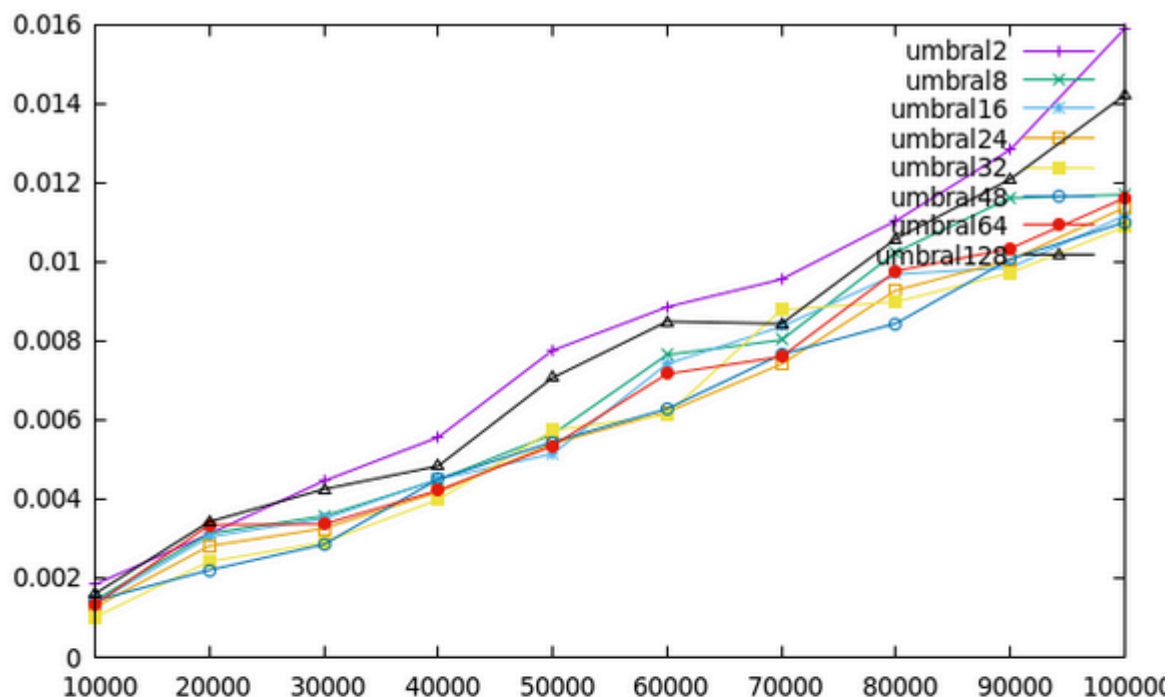
Esta gráfica muestra el comportamiento del algoritmo Quicksort al variar el umbral utilizado para aplicar el algoritmo de inserción en subproblemas pequeños, trabajando con strings de tamaño reducido. Se observa que, aunque las diferencias no son extremadamente grandes, existe una ligera tendencia a que los umbrales más altos presenten peores tiempos de ejecución. Esto se debe a que, al aumentar el umbral, el algoritmo de inserción se aplica sobre subproblemas de mayor tamaño, introduciendo un coste cuadrático que, aunque limitado, resulta apreciable en problemas de tamaño pequeño. Por el contrario, los umbrales más bajos permiten que Quicksort gestione la mayor parte del proceso mediante particiones, lo que resulta más eficiente.



A diferencia de lo observado en tamaños pequeños, en este caso las curvas correspondientes a los distintos umbrales prácticamente coinciden, mostrando tiempos de ejecución muy similares. Este comportamiento indica que, para tamaños grandes, la influencia del umbral es mínima en el rendimiento global del algoritmo. La razón principal es que el coste dominante viene determinado por la complejidad media de Quicksort, $O(n \log n)$, mientras que el algoritmo de inserción sólo se aplica sobre subproblemas de tamaño reducido, teniendo un impacto poco significativo en el tiempo total. Por tanto, aunque la elección del umbral puede afectar ligeramente a la constante del algoritmo, su influencia se diluye a medida que el tamaño del problema crece. En consecuencia, para grandes volúmenes de datos, todos los umbrales considerados ofrecen un rendimiento prácticamente equivalente.

En conclusión, mientras que en Mergesort la elección del umbral es un factor crítico que afecta de forma notable al rendimiento, en Quicksort su influencia es mucho más limitada, especialmente en problemas de gran tamaño.

Estudio de los distintos umbrales para Mergesort con ints:



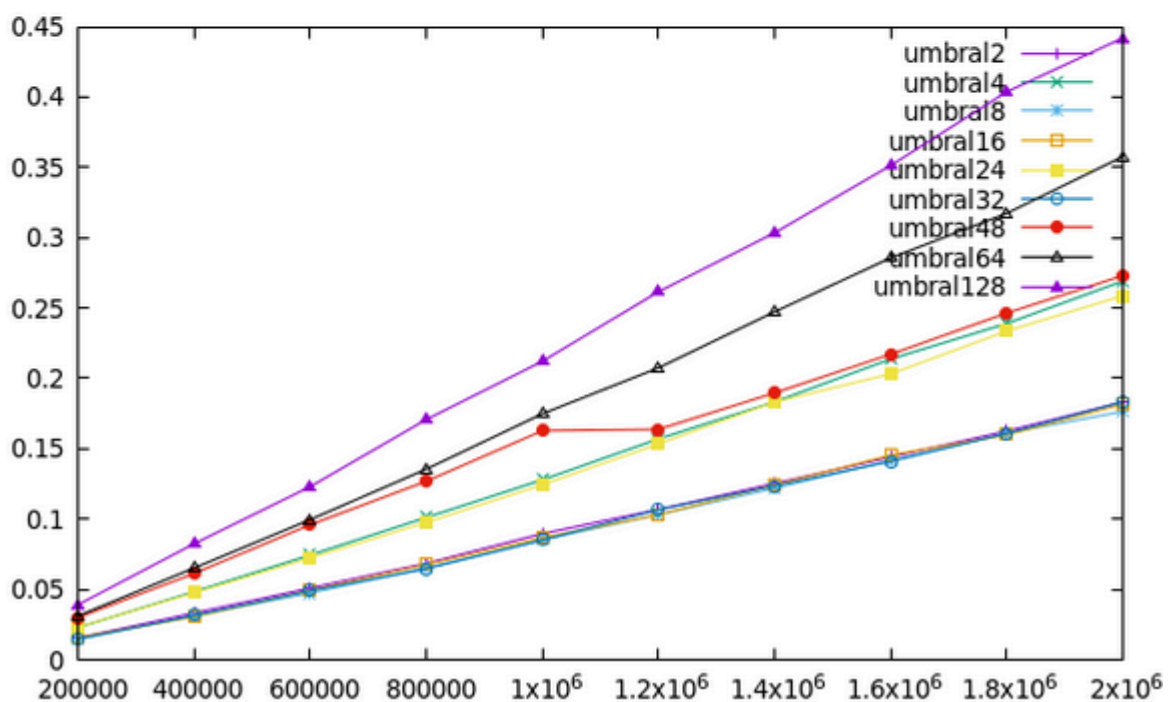
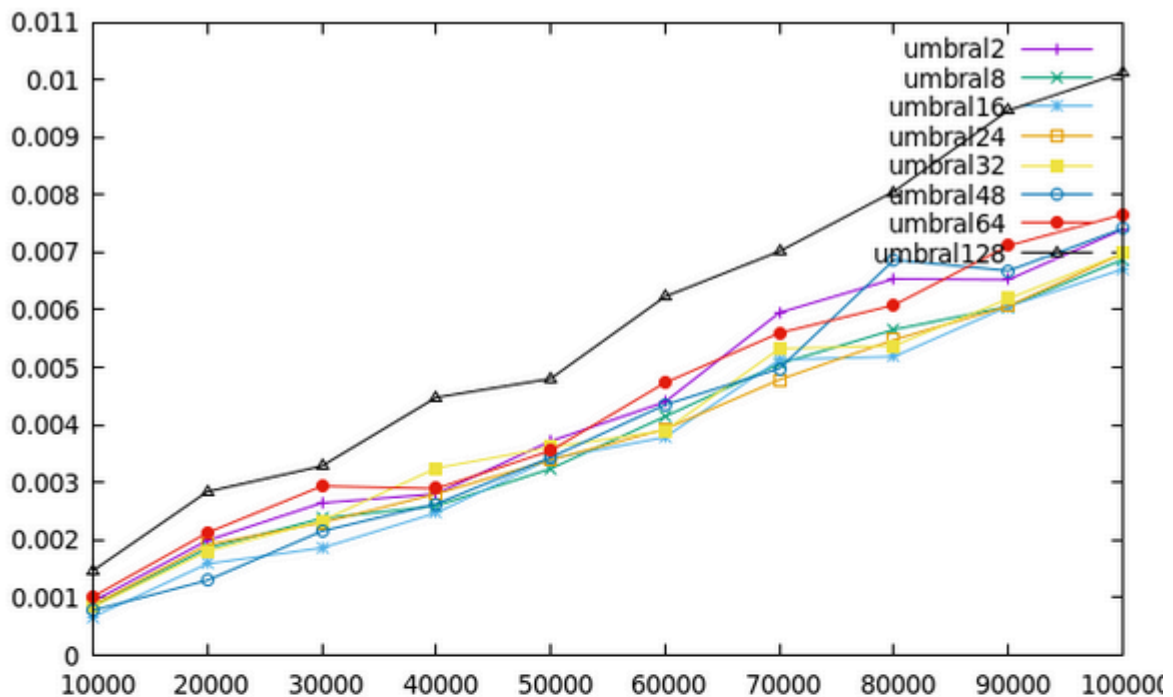
Esta gráfica muestra el comportamiento del algoritmo Mergesort al aplicar distintos umbrales utilizando datos de tipo entero. Se observa que, en general, las diferencias entre los distintos umbrales son relativamente pequeñas, manteniéndose todas las curvas bastante próximas entre sí.

No obstante, se aprecia que los valores extremos, especialmente los umbrales 2 y 128, presentan los peores tiempos de ejecución. En el caso del umbral 2, el algoritmo apenas utiliza el método de inserción, lo que provoca que Mergesort realice un mayor número de llamadas recursivas y operaciones de mezcla, introduciendo una sobrecarga innecesaria.

Por otro lado, el umbral 128 implica que el algoritmo de inserción se aplica sobre subproblemas demasiado grandes, lo que introduce un coste cuadrático que penaliza el rendimiento.

Por el contrario, los mejores resultados se obtienen con umbrales intermedios, concretamente entre 24 y 48, donde se alcanza un equilibrio adecuado entre el uso del algoritmo de inserción y la estrategia de divide y vencerás.

Estudio de los distintos umbrales para Quicksort con ints:



Esta gráfica muestra el comportamiento del algoritmo Quicksort al variar el umbral para la aplicación del algoritmo de inserción con datos de tipo entero. Se observa que, en general, las diferencias entre la mayoría de los umbrales son poco significativas, manteniéndose las curvas bastante próximas entre sí. Sin embargo, destaca claramente el umbral 128 como el peor de todos, presentando tiempos de ejecución superiores al resto.

En problemas de mayor tamaño, la gráfica muestra cómo las diferencias entre los distintos umbrales se hacen mucho más evidentes. A medida que aumenta el tamaño del problema, las curvas se separan de forma clara, permitiendo identificar con mayor precisión el impacto de la elección del umbral en el rendimiento del algoritmo. Los umbrales intermedios, como 48 y 24, presentan un comportamiento más equilibrado, situándose en una posición intermedia en cuanto a rendimiento. Sin embargo, los mejores resultados se obtienen con los umbrales más bajos e intermedios (2, 8, 16 y 32), cuyas curvas prácticamente coinciden. En Mergesort, el rendimiento óptimo se alcanza con valores intermedios del umbral (entre 24 y 48), ya que permiten equilibrar adecuadamente el coste de la recursión y el uso del algoritmo de inserción. En Quicksort se observa una mayor tolerancia en la elección del umbral. Los mejores resultados se obtienen en un rango más amplio de valores, concretamente entre 2 y 32, donde las diferencias son prácticamente despreciables incluso en tamaños grandes.

A partir de los resultados obtenidos en el estudio empírico, se puede concluir que el tipo de dato utilizado tiene una influencia determinante en el impacto del umbral sobre el rendimiento de los algoritmos de ordenación. En el caso de los strings, el umbral juega un papel mucho más crítico. Esto se debe a que tanto las comparaciones como el movimiento de datos son operaciones costosas, especialmente cuando las cadenas son largas o comparten prefijos comunes. Como consecuencia, el uso del algoritmo de inserción en subproblemas de tamaño moderado introduce una penalización significativa, lo que hace que los umbrales bajos (2 y 4) sean claramente los más eficientes. En este contexto, una mala elección del umbral provoca diferencias muy acusadas en los tiempos de ejecución, especialmente en tamaños grandes. Por el contrario, al trabajar con enteros, el impacto del umbral es considerablemente menor. Las operaciones sobre enteros son mucho más eficientes, tanto en comparación como en copia, lo que reduce el coste del algoritmo de inserción incluso cuando se aplica a subproblemas de mayor tamaño. Esto se traduce en curvas mucho más próximas entre sí y en una mayor tolerancia en la elección del umbral.